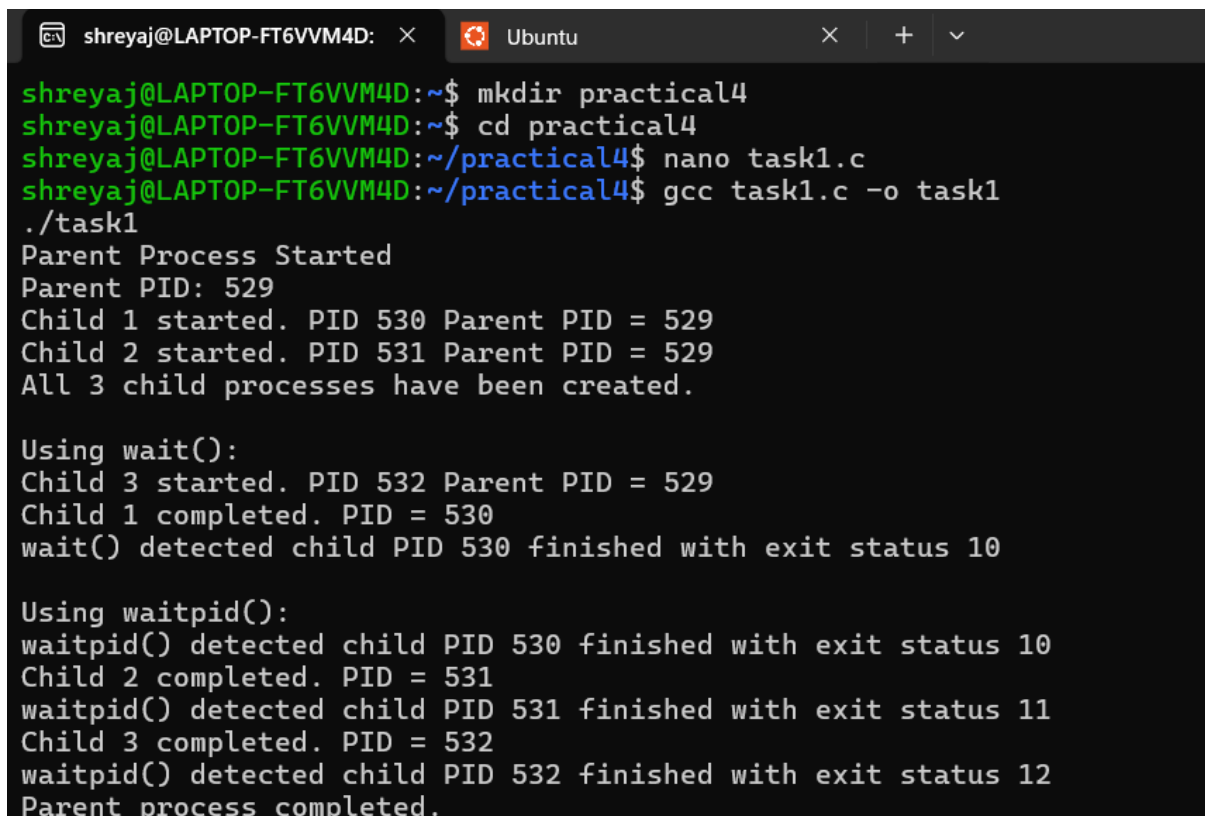


PRACTICAL WEEK – 4

Task 1 : Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.



```
shreyaj@LAPTOP-FT6VVM4D: ~$ mkdir practical4
shreyaj@LAPTOP-FT6VVM4D: ~$ cd practical4
shreyaj@LAPTOP-FT6VVM4D: ~/practical4$ nano task1.c
shreyaj@LAPTOP-FT6VVM4D: ~/practical4$ gcc task1.c -o task1
./task1
Parent Process Started
Parent PID: 529
Child 1 started. PID 530 Parent PID = 529
Child 2 started. PID 531 Parent PID = 529
All 3 child processes have been created.

Using wait():
Child 3 started. PID 532 Parent PID = 529
Child 1 completed. PID = 530
wait() detected child PID 530 finished with exit status 10

Using waitpid():
waitpid() detected child PID 530 finished with exit status 10
Child 2 completed. PID = 531
waitpid() detected child PID 531 finished with exit status 11
Child 3 completed. PID = 532
waitpid() detected child PID 532 finished with exit status 12
Parent process completed.
```

```
shreyaj@LAPTOP-FT6VVM4D: x Ubuntu task1.c
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid[3];
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    for (int i = 0; i < 3; i++) {
        pid[i] = fork();

        if (pid[i] == 0) {
            printf("Child %d started. PID %d Parent PID = %d\n", i + 1, getpid(), getppid());
            sleep(i + 1);
            printf("Child %d completed. PID = %d\n", i + 1, getpid());
            exit(10 + i);
        }
    }

    printf("All 3 child processes have been created.\n");

    printf("\nUsing wait():\n");
    pid_t finished = wait(&status);
    printf("wait() detected child PID %d finished with exit status %d\n", finished, WEXITSTATUS(status));

    printf("\nUsing waitpid():\n");
    for (int i = 0; i < 3; i++) {
        waitpid(pid[i], &status, 0);
        printf("waitpid() detected child PID %d finished with exit status %d\n", pid[i], WEXITSTATUS(status));
    }

    printf("Parent process completed.\n");
    return 0;
}
```

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
GNU nano 7.2 task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process terminating...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping...\n");
        sleep(30);
        printf("Parent terminating...\n");
    }

    return 0;
}
```

```

shreyaj@LAPTOP-FT6VVM4D:~/practical4$ nano task2.c
shreyaj@LAPTOP-FT6VVM4D:~/practical4$ gcc task2.c -o task2
./task2
Parent process: PID = 566
Child PID = 567
Parent sleeping...
Child process: PID = 567
Child process terminating...
Parent terminating...
shreyaj@LAPTOP-FT6VVM4D:~/practical4$ nano task2.c
shreyaj@LAPTOP-FT6VVM4D:~/practical4$ gcc task2.c -o task2
./task2
Child process terminating...
Zombie process removed.
shreyaj@LAPTOP-FT6VVM4D:~/practical4$ nano task1.c
shreyaj@LAPTOP-FT6VVM4D:~/practical4$ nano task2.c
shreyaj@LAPTOP-FT6VVM4D:~/practical4$ |

```

```

shreyaj@LAPTOP-FT6VVM4D: $ ps -el | grep task2
0 S 1000 566 317 0 80 0 - 692 hrttime pts/0 00:00:00 task2
1 Z 1000 567 566 0 80 0 - 0 - pts/0 00:00:00 task2
shreyaj@LAPTOP-FT6VVM4D: $

```

```

GNU nano 8.7.1 task2.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child process terminating...\n");
        exit(0);
    } else {
        wait(NULL);
        printf("Zombie process removed.\n");
    }

    return 0;
}

```